

## ของที่ระลึก

อมารุกำลังซื้อของที่ระลึกในร้านค้าต่างประเทศ ของที่ระลึกมีอยู่  $N$  ชนิด โดยร้านค้ามีของที่ระลึกแต่ละชนิดเป็นจำนวนไม่จำกัด

ของที่ระลึกแต่ละชนิดมีราคาคงที่ หมายความว่า ของที่ระลึกชนิดที่  $i$  ( $0 \leq i < N$ ) จะมีราคาเท่ากับ  $P[i]$  เหรียญ โดย  $P[i]$  เป็นจำนวนเต็มบวก

อมารุทราบว่าของที่ระลึกถูกเรียงหมายเลขชนิดตามราคาจากมากไปน้อย และของที่ระลึกแต่ละชนิดมีราคาแตกต่างกัน กล่าวอย่างชัดเจนคือ  $P[0] > P[1] > \dots > P[N-1] > 0$ , นอกจากนั้นอมารุรู้ราคา  $P[0]$  อย่างไรก็ตามอมารุไม่มีข้อมูลอื่น ๆ เกี่ยวกับราคาของของที่ระลึก

ในการซื้อของที่ระลึก อมารุจะต้องทำธุรกรรมจำนวนหนึ่งกับผู้ขาย

การทำธุรกรรมแต่ละครั้งประกอบด้วยขั้นตอนดังต่อไปนี้:

- อมารุให้เหรียญจำนวนหนึ่ง (เป็นจำนวนบวก) กับผู้ขาย
- ผู้ขายวางเหรียญเหล่านี้รวมกันเป็นกองเดียวบนโต๊ะในห้องด้านหลัง ซึ่งอมารุไม่สามารถมองเห็นได้
- ผู้ขายพิจารณาของที่ระลึกแต่ละชนิด  $0, 1, \dots, N-1$  ตามลำดับที่ละชนิด แต่ละชนิดจะถูกพิจารณา **หนึ่งครั้งเท่านั้น** ต่อหนึ่งธุรกรรม
  - เมื่อพิจารณาของที่ระลึกชนิดที่  $i$  หากจำนวนเหรียญปัจจุบันในกองมีอย่างน้อย  $P[i]$  ดังนั้น
    - ผู้ขายนำเหรียญ  $P[i]$  ออกจากกอง และ
    - ผู้ขายวางของที่ระลึกชนิดที่  $i$  หนึ่งชิ้นไว้บนโต๊ะ
- ผู้ขายทอนเหรียญทั้งหมดที่เหลืออยู่ในกองและนำของที่ระลึกทั้งหมดบนโต๊ะให้กับอมารุ

สังเกตว่าไม่มีเหรียญหรือของที่ระลึกบนโต๊ะก่อนเริ่มทำธุรกรรม

งานของคุณคือการสั่งให้อมารุทำธุรกรรมจำนวนหนึ่ง โดย:

- ในการทำธุรกรรมแต่ละครั้งเขาจะซื้อของที่ระลึก **อย่างน้อยหนึ่งชิ้น** และ
- โดยรวมแล้วเขาซื้อของที่ระลึกชนิดที่  $i$  จำนวน  $i$  **ชิ้นพอดี** สำหรับแต่ละ  $i$  ที่  $0 \leq i < N$  สังเกตว่าตามเงื่อนไขนี้ อมารุไม่ควรจะซื้อของที่ระลึกชนิดที่ 0

อมารุไม่จำเป็นต้องทำให้จำนวนธุรกรรมน้อยที่สุด และเขามีจำนวนเงินที่จะใช้ซื้ออยู่ไม่จำกัด

## รายละเอียดการเขียนโปรแกรม

คุณจะต้องเขียนฟังก์ชันต่อไปนี้

```
void buy_souvenirs(int N, long long P0)
```

- $N$ : เป็นจำนวนชนิดของของที่ระลึก
- $P0$ : เป็นค่าของ  $P[0]$
- ฟังก์ชันนี้จะถูกเรียกหนึ่งครั้งเท่านั้นสำหรับแต่ละกรณีทดสอบ

ฟังก์ชันด้านบนสามารถเรียกใช้ฟังก์ชันต่อไปนี้เพื่อสั่งให้มารุทำธุรกรรม

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- $M$ : จำนวนเหรียญที่มารุให้ผู้ขาย
- ฟังก์ชันต้องคืนค่าเป็น pair ซึ่งมีสมาชิกตัวแรกเป็นอาร์เรย์  $L$  ประกอบด้วยชนิดของของที่ระลึกที่ถูกซื้อ (เรียงลำดับตามชนิดของของที่ระลึกจากน้อยไปมาก) สมาชิกตัวที่สองเป็นค่าจำนวนเต็ม  $R$  ซึ่งเป็นจำนวนเหรียญที่ทอนให้มารุหลังจากการทำธุรกรรม
- มีความจำเป็นที่  $P[0] > M \geq P[N - 1]$  โดยเงื่อนไข  $P[0] > M$  รับประกันว่ามารุจะไม่ซื้อของที่ระลึกชนิดที่ 0 และเงื่อนไข  $M \geq P[N - 1]$  รับประกันว่ามารุจะซื้อของที่ระลึกอย่างน้อยหนึ่งชิ้นเสมอ หากไม่เป็นไปตามเงื่อนไขเหล่านี้ ไซลูชันของคุณจะได้รับคำตัดสินว่า Output isn't correct: Invalid argument ` สังเกตว่าค่าของ  $P[N - 1]$  นั้นจะไม่ได้ระบุไว้ในข้อมูลนำเข้า ซึ่งแตกต่างจากค่า  $P[0]$  ที่โจทย์ระบุมาให้
- สามารถเรียกฟังก์ชันดังกล่าวได้มากที่สุด 5000 ครั้งในแต่ละกรณีทดสอบ

เทรดเดอร์ตัวอย่างจะไม่ทำงานแบบปรับเปลี่ยนได้ (ไม่ adaptive) ซึ่งแปลว่าลำดับของราคา  $P$  ไม่เปลี่ยนแปลงหลังจาก buy\_souvenirs ถูกเรียก

## เงื่อนไข

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$  สำหรับแต่ละ  $i$  ที่  $0 \leq i < N$
- $P[i] > P[i + 1]$  สำหรับแต่ละ  $i$  ที่  $0 \leq i < N - 1$

## ปัญหาย่อย

| ปัญหาย่อย | คะแนน | เงื่อนไขเพิ่มเติม                                                                                                                             |
|-----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1         | 4     | $N = 2$                                                                                                                                       |
| 2         | 3     | $P[i] = N - i$ สำหรับแต่ละ $i$ ที่ $0 \leq i < N$                                                                                             |
| 3         | 14    | $P[i] \leq P[i + 1] + 2$ สำหรับแต่ละ $i$ ที่ $0 \leq i < N - 1$                                                                               |
| 4         | 18    | $N = 3$                                                                                                                                       |
| 5         | 28    | $P[i + 1] + P[i + 2] \leq P[i]$ สำหรับแต่ละ $i$ ที่ $0 \leq i < N - 2$<br>$P[i] \leq 2 \cdot P[i + 1]$ สำหรับแต่ละ $i$ ที่ $0 \leq i < N - 1$ |
| 6         | 33    | ไม่มีเงื่อนไขเพิ่มเติม                                                                                                                        |

## ตัวอย่าง

พิจารณาการเรียกฟังก์ชันดังนี้

```
buy_souvenirs(3, 4)
```

มีของที่ระลึก  $N = 3$  ชนิดและ  $P[0] = 4$ . สังเกตว่ามีโอกาสเดียวของลำดับที่เป็นไปได้ของราคา  $P$ :  $[4, 3, 2]$ ,  $[4, 3, 1]$ , และ  $[4, 2, 1]$

สมมติว่า `buy_souvenirs` เรียก `transaction(2)` สมมติว่าฟังก์ชันคืนค่า  $([2], 1)$  แปลว่าอमारูซื้อของที่ระลึกชนิดที่ 2 และผู้ขายคืนเหรียญ 1 เหรียญ สังเกตว่าการคืนค่านี้ทำให้เราอนุมานได้ว่า  $P = [4, 3, 1]$ , ดังนั้น:

- สำหรับ  $P = [4, 3, 2]$ , `transaction(2)` จะต้องคืนค่า  $([2], 0)$
- สำหรับ  $P = [4, 2, 1]$ , `transaction(2)` จะต้องคืนค่า  $([1], 0)$

ดังนั้น `buy_souvenirs` สามารถเรียก `transaction(3)` ซึ่งคืนค่า  $([1], 0)$ , แปลว่าอमारูซื้อของที่ระลึกชนิดที่ 1 หนึ่งชิ้น และผู้ขายคืนเงินให้ 0 เหรียญ จนถึงตอนนี้อमारูได้ซื้อของที่ระลึกชนิดที่ 1 หนึ่งชิ้น และ ของที่ระลึกชนิดที่ 2 หนึ่งชิ้น

ในที่สุด `buy_souvenirs` สามารถเรียก `transaction(1)`, ซึ่งคืนค่า  $([2], 0)$ , แปลว่าอमारูซื้อของที่ระลึกชนิดที่ 2 สังเกตว่าเราสามารถเรียก `transaction(2)` ได้ ถึงเวลานี้อमारูได้ซื้อของที่ระลึกชนิดที่ 1 หนึ่งชิ้น และ ของที่ระลึกชนิดที่ 2 สองชิ้น ตามที่กำหนด

## เกรดเดอร์ตัวอย่าง

รูปแบบข้อมูลนำเข้า:

```
N
P[0] P[1] ... P[N-1]
```

รูปแบบข้อมูลส่งออก:

```
Q[0] Q[1] ... Q[N-1]
```

กรณีนี้  $Q[i]$  เป็นจำนวนของของที่ระลึกชนิดที่  $i$  ที่ถูกซื้อทั้งหมดสำหรับแต่ละ  $i$  ที่  $0 \leq i < N$